Open in app ↗ Sign up Sign in

Medium Search

Обзор книги "Accelerate" (part II)

Alexander Polomodov · Follow
5 min read · Aug 3, 2023

Share

В этой статье я продолжу разбирать классическую книгу "`Accelerate`", которая так нравится менеджерам за понятные советы о развитии `capabilities` для повышения эффективности организации. В прошлой статье мы поговорили про

— Общую информацию об исследовании и основные выводы
— Способы измерения `performance` разработки, как плохие, так и хорошие
— Подходы к типологиям культур и их влияние на распространение информации и эффективность работы организации

В этой статье пойдет речь про технические практики, архитектуру и интеграцию вопросов безопасности в процессы разработки.

Рис.1 "Содержимое статьи"

4. Technical practices

Эта глава начинается с экскурса в прошлое, а именно в момент принятия `Agile Manifesto`, во времена которого самой популярным `Agile` фреймворком было `XP` (`eXtreme Programming`), где большой упор делался на инженерных практиках (`TDD` и `Continuous Delivery`, хотя это так тогда и не называлось). Но прошло время и по мере адаптации `Agile` на передний план вышли фреймворки, в которых менеджерские и технические практики вышли на первые роли, а инженерные практики стали вторичными. В данном исследовании показано, что технические практики играют жизненно важную роль в достижении результатов.

?

что для

Дальше авторы переходят к обсуждению ключевых принципов `Devops`, которые изложены на рисунке ниже.

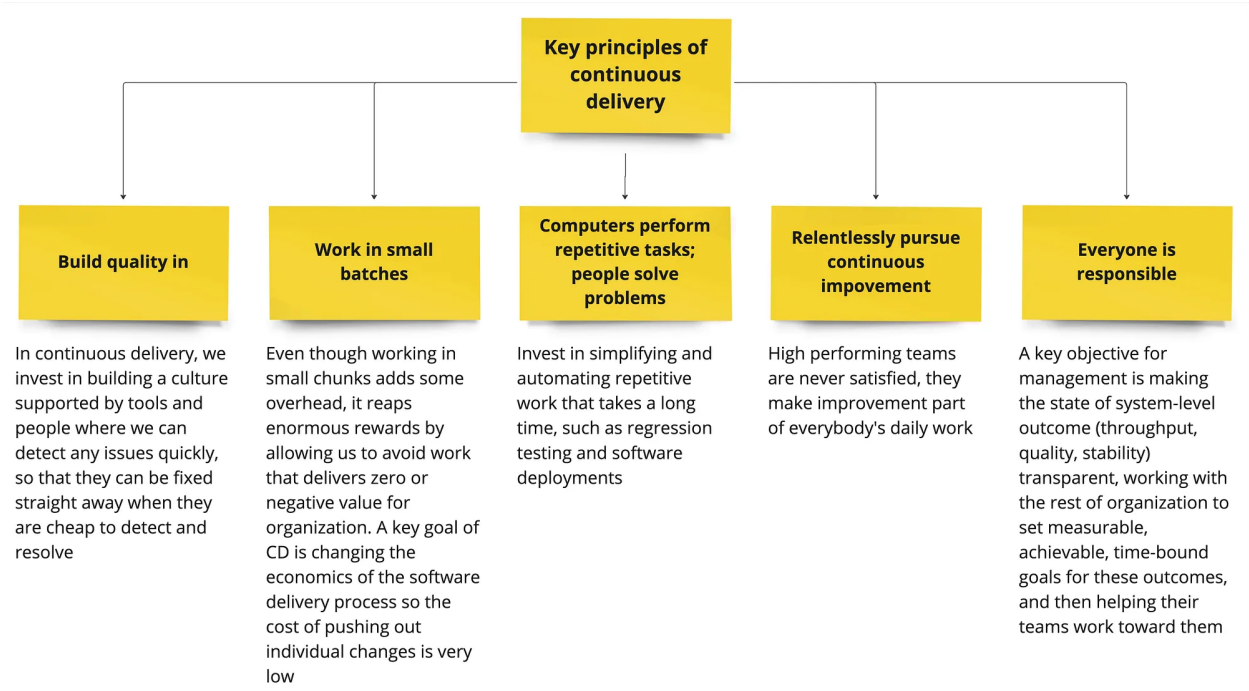


Рис.2 "Key principles of continuous delivery"

Дальше авторы показывают три основы `continuous delivery`

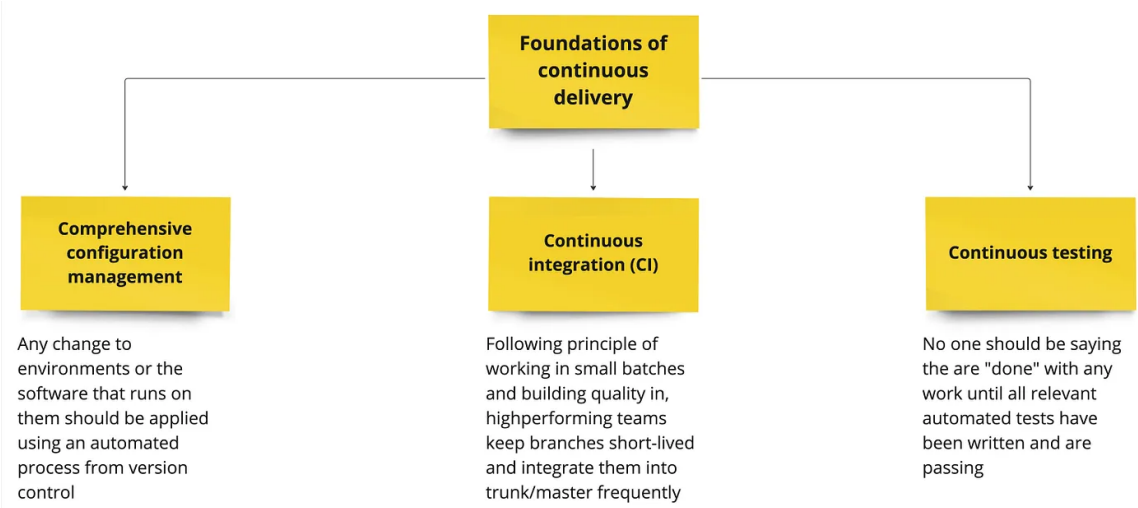


Рис.3 "Foundations of continuous delivery"

В итоге, авторы показывают какие факторы из их исследования являются драйверам `continuous delivery` и на что влияет сам `CD`

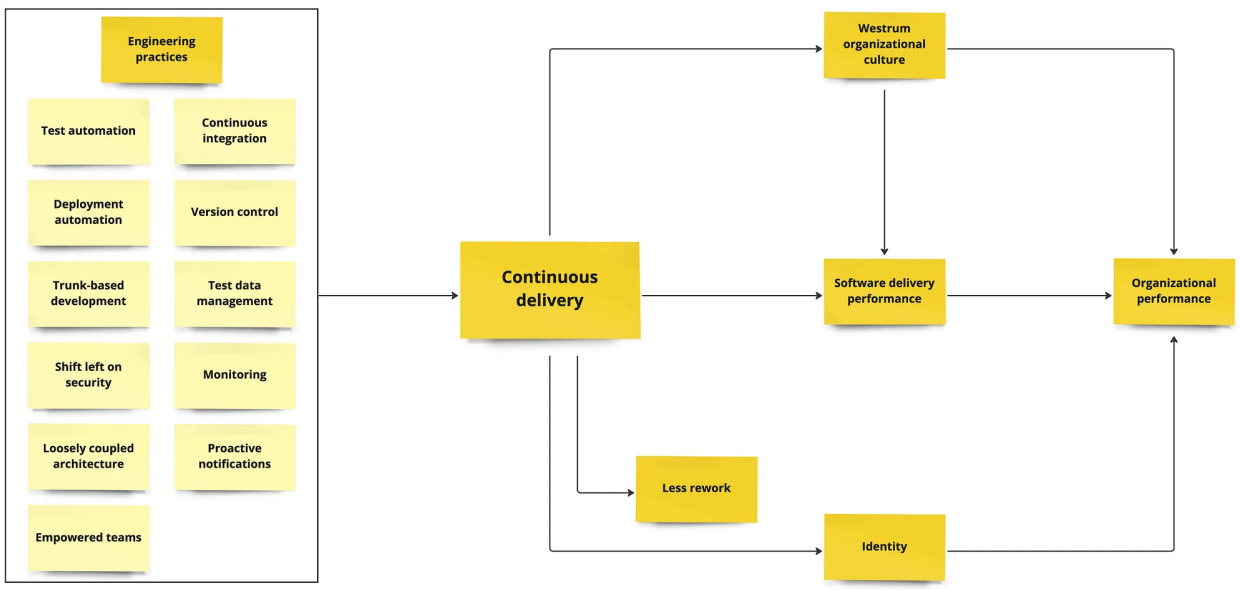


Рис.4 "Drivers and impacts of continuous delivery"

Как мы видим для `continuous delivery` важны инженерные практики, а сам `continuous delivery` нужен для повышения эффективности разработки, создания и поддержания правильной культуры и как результат повышения эффективности организации.

Дальше в главе авторы рассматривают часть инженерных практик, о которых дальше они не планируют говорить, например

— `version control` — базовая история про использование системы контроля версий для кода, конфигураций и всего остального
— `test automation` — про автоматизацию тестирования
— `test data management` — про управление данными для прогона тестов

что это значит?

— TBD (trunk-based development) — подход для работы с кодом, который пришел на смену git-flow и остальным, когда у нас нет долгоживущих веток и все изменения быстро и оперативно попадают в trunk / master . В этом случае зачастую надо уметь использовать feature toggles / flags для закрытия той логики, что еще не должна быть доступна пользователям продукта
— information security — security должна быть встроена в процесс разработки, а не появляться внезапно в конце процесса в форме шлагбаума, блокирующего выпуск. Про это интересно рассказано в книге " Agile Application Security: Enabling Security in a Continuous Delivery Pipeline ", про которую я рассказывал отдельно.

А дальше авторы переходят к теме архитектуры

5. Architecture

В этой главе авторы подчеркивают значимость и влияние архитектуры на темп разработки и стабильность получающихся систем. Дальше авторы рассматривают разные типы систем и их связь с delivery performance . Авторы рассматривали следующие типы приложений

— greenfield — приложения, у которых еще не было релиза
— system of engagement — приложения, с которыми взаимодействуют конечные пользователи
— system of records — приложения с критичной бизнес-информацией, для которых критична консистентность данных
— custom software разработанное на заказ и внутри компании
— COTS (commercial off-the-shelf) — коробочные решения
— встроенные приложения
— мобильные приложения
— приложения под мейнфреймы

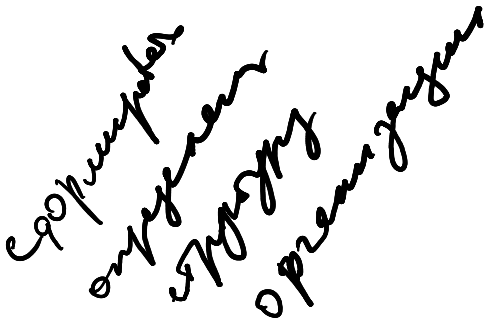
В итоге, авторы не нашли связи между типом приложений и `delivery performance`. Но они нашли такую связь между двумя архитектурными характеристиками и эффективностью `delivery` — эти характеристики

— `testability` — приложения можно протестировать без интеграционного контура
— `deployability` — приложения можно разворачивать независимо друг от друга

По-факту, для достижения этих характеристик нужно определенным образом проектировать программное обеспечение, чтобы получилась `loosely-coupled architecture`. Дальше авторы вспоминают закон Конвея

> *Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure*

И обратный маневр Конвея, где мы организуем команды так, чтобы получить желаемую архитектуру, которая помогает командам выполнять свою работу целиком от дизайна до деплоя системы без масштабных коммуникаций с другими командами.



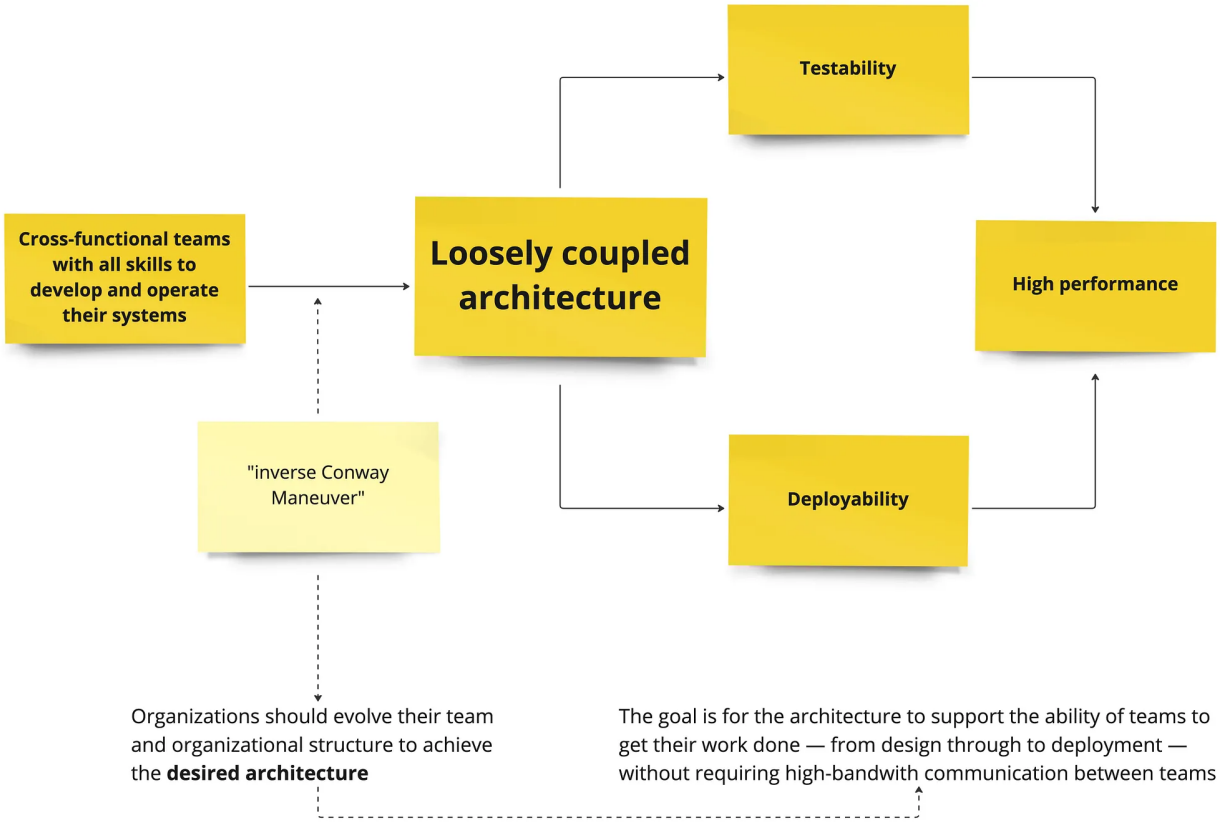


Рис.5 "Architecture"

Для этого можно использовать подходы из `DDD` (`domain driven design`) с `bounded context` и `API`, которые можно использовать как путь для декомпозиции крупных доменов в маленькие, менее связные части, которые можно разрабатывать, тестировать и деплоить отдельно. Подробнее про эти концепции рекомендую посмотреть книгу "`Learning DDD`", которую мы обсуждали в клубе "`Code of Architecture`". Также про дизайн систем очень круто написано в книге "`A Philosophy of Software Design`", что мы обсуждали в этом же клубе.

Дальше авторы показывают, что правильная архитектура помогает организации масштабироваться, сохраняя производительность.

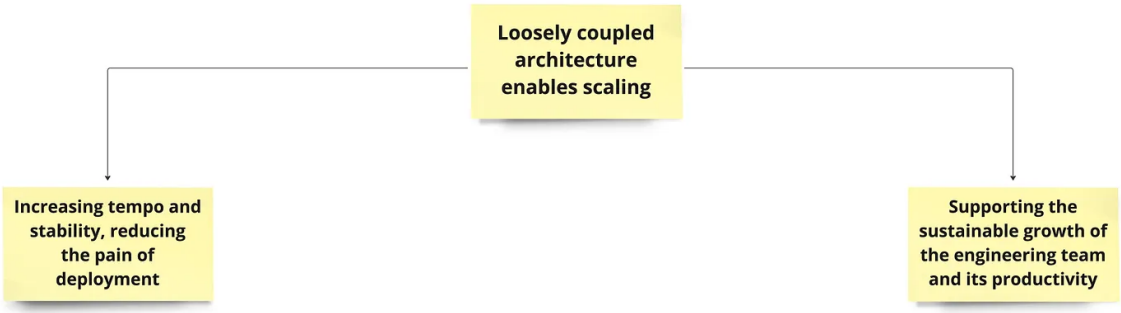


Рис.6 "Loosely coupled architecture enables scaling"

Я рассказывал про такое масштабирование в статье " 'Канал. Продукт. Платформа' или эволюция подходов к развитию мобильного банка Тинькофф". Плюс была такая интересная `whitepaper` про `continuous delivery` в одном из бигтехов и как они масштабировали свою разработку, которую я разбирал в отдельной статье.

Дальше авторы говорят про выбор инструментов и про место для стандартизации или произвольный выбор элементов. Интересно, что глубже эта тема разбирается в книге " `Team topologies` ", что была издана в 2019 году, что на год позже " `Accelerate` ". Я разбирал эту книгу в трех статьях

— `Teams as means of Delivery`

— `Team Topologies that work for flow`

— `Evolving team interactions for innovation and rapid delivery`

В этой книге вводится топология команд, где основными являются платформенные команды, которые делают инструменты, используемые продуктовыми (`stream-aligned`) командами для создания продуктов как для конечных, так и для внутренних пользователей. Причем эти инструменты тоже являются продуктами, которые работают в концепции `dev2dev` решений.

В финале этой главы авторы говорят золотые слова

> *Architects should focus on engineers and outcomes, not tools or technologies*

6. Integrating infosec into the delivery lifecycle

На эту тему в 2017 году вышла отличная книга "`Agile Application Security`", про которую я рассказывал отдельно. В книге "`Accelerate`" делается фокус на `shift-left security`, который влечет за собой следующие моменты.

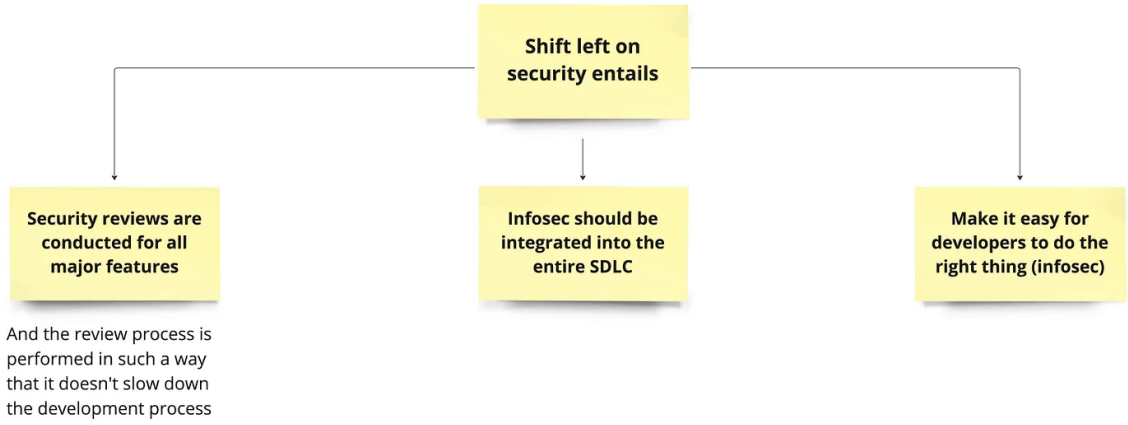


Рис.7 "Shift left on security"

В следующей части мы продолжим разбирать содержимое книги и поговорим про практики менеджмента и лидерства в разработке программного обеспечения.

Delivery Management | Culture | Management | DevOps | Architecture